

Notas del curso de modelo lineal

Modelo Lineal Bayesiano

Humberto Vaquera Huerta
2024

Introducción

La aplicación de modelos lineales bayesianos sigue el esquema general de la estadística bayesiana: Definir la distribución *a priori* correspondiente para los parámetros.

- Determinar la verosimilitud de los datos
- Aplicar el teorema de Bayes para actualizar la distribución a priori en forma de distribución *a posteriori*.

Programas de computacionales para realizar Modelos Lineales Bayesianos:

- WinBugs, OpenBugs, Jags
- Stan
- SAS proc MCMC
- R

El modelo lineal Gauss Markov

Se define como:

$$\mathbf{Y} = \mathbf{X}\beta + \varepsilon$$

Donde:

$$\varepsilon \sim N(0, \sigma^2 I)$$

Es decir, los errores se extraen independientemente de una distribución normal con media 0 y varianza σ^2

- $\mathbf{X}$ es la matriz de diseño
- β es el vector de coeficientes
- I es la matriz de identidad: diagonales todas 1, fuera de las diagonales todas 0.

Esto permite la extensión a múltiples variables y a variables categóricas, como veremos a continuación.

Note que I significa que no hay correlación entre los errores.

El modelo lineal Gauss Markov

Del modelo anterior se tiene que :

$$\mathbf{Y} \sim N(\boldsymbol{\beta}, \sigma^2 \mathbf{I})$$

los estimadores frecuentistas de $\boldsymbol{\beta}$ son:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^t \mathbf{X})^{-1} (\mathbf{X}^t \mathbf{Y})$$

Por el teorema de Gauss-Markov este estimador es el MELI (BLUE).

El estimador frecuentista de σ^2 es $s^2 = (\mathbf{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})^t (\mathbf{Y} - \mathbf{X}\hat{\boldsymbol{\beta}}) / (n - k)$ con $k = \text{rango}(X)$ y n =numero de obs.

La matriz de varianzas y covarianzas de $\hat{\boldsymbol{\beta}}$ es

$$\text{Var}(\hat{\boldsymbol{\beta}}) = s^2 (\mathbf{X}^t \mathbf{X})^{-1}$$

Podemos usar V_i elemento diagonal de $\text{Var}(\hat{\boldsymbol{\beta}})$, por lo que el estadístico $(\beta_i - 0) / (s(V_i)^{1/2})$ lo usamos para probar hipótesis $H_0 : \beta_i = 0$.

Estimacion Bayesiana en el modelo lineal

Para el modelo

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

$$\boldsymbol{\varepsilon} \sim N(0, \sigma^2 I)$$

El objeto de la inferencia bayesiana es la distribución posterior de vector de parámetros $\boldsymbol{\beta}$ y del parámetro σ^2).

Usando la regla de Bayes, sabemos que esto es:

$$f(\boldsymbol{\beta}, \sigma^2 | \mathbf{Y}, \mathbf{X}) \propto \pi(\boldsymbol{\beta}, \sigma^2) \times L(\boldsymbol{\beta}, \sigma^2 | Y)$$

donde : la verosimilitud

$$L(\boldsymbol{\beta}, \sigma^2 | Y) = \prod_{j=1}^n f(y_j; \boldsymbol{\beta}, \sigma^2)$$

Estimacion Bayesiana en el modelo lineal

$$\begin{aligned} &= \prod_{j=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{1}{2\sigma^2} (y_j - f(x_j, \beta))^2 \right) \\ &= \frac{1}{(2\pi\sigma^2)^{n/2}} \exp \left(-\frac{1}{2\sigma^2} \sum_{j=1}^n (y_j - f(x_j, \beta))^2 \right). \end{aligned}$$

y

$$\pi(\beta, \sigma^2) = \pi(\beta) \times \pi(\sigma^2)$$

es la distribución a priori de β, σ^2

Priori

Para realizar un análisis bayesiano completo utilizando la probabilidad dada anteriormente, consideramos una distribución previa semiconjugada para β y σ^2 de la forma

$$\beta \sim \mathbf{N}(\beta_0, \Sigma_0) \quad \text{y} \quad \sigma^2 \sim \text{IG}(\nu_0/2, \nu_0\sigma_0^2/2)$$

A falta de información externa convincente a los datos, se acostumbra utilizar una **distribución previa** que sea lo menos informativa posible. Kass y Wasserman (1995), dejamos con $g = nm$ y $\nu_0 = 1$, y establecemos $\beta_0 = \hat{\beta}_{\text{OLS}}$, $\Sigma_0 = n\sigma_0^2(X^t X)^{-1}$, con $\sigma_0^2 = \hat{\sigma}_{\text{OLS}}^2$, donde $\hat{\phi}_{\text{OLS}}$ representa para la estimación de mínimos cuadrados ordinarios (MCO) de ϕ .

Esta elección de g hace que la relación $\frac{g}{g+1}$ sea muy cercana a 1 y, por lo tanto, β está prácticamente centrada alrededor de 0; de manera similar, la distribución previa de σ^2 está débilmente centrada alrededor de $\hat{\sigma}_{\text{OLS}}^2$ ya que $\nu_0 = 1$. Tal distribución previa no puede considerarse estrictamente una distribución previa real, ya que requiere conocimiento de Y para ser construida. Sin embargo, sólo utiliza una pequeña cantidad de la información proporcionada en Y , y puede considerarse vagamente como la distribución previa de un investigador con información previa imparcial pero débil.

Inferencia posterior

La distribución posterior se puede explorar utilizando métodos de cadena de Markov Monte Carlo (MCMC) como el muestreador de Gibbs. Implementar un algoritmo de este tipo en el modelo anterior es bastante simple ya que las distribuciones condicionales completas son

$$\beta|y, X, \sigma^2 \sim \mathbf{N}_p((\Sigma_0^{-1} + \sigma^{-2} X^\top X)^{-1}(\Sigma_0^{-1}\beta_0 + \sigma^{-2} X^\top y), (\Sigma_0^{-1} + \sigma^{-2} X^\top X)^{-1})$$

y

$$\sigma^2 \mid y, X, \beta \sim \text{IG}((\nu_0 + nm)/2, (\nu_0\sigma_0^2 + (y - X\beta)^\top (y - X\beta))/2).$$

Vear *christensen 2011 Bayesian Ideas and Data Analysis: An Introduction for Scientists and Statisticians*. para obtener detalles.

Estimacion Bayesiana en el modelo lineal

En el contexto Bayesiano, necesitamos especificar distribuciones a priori para β y $\tau = 1/\sigma^2$ (precisión): $\pi(\beta)$ y $\pi(\sigma^2)$, independientes. Por lo tanto, la distribución *a posteriori* de los parámetros en el modelo está dada por:

$$p(\beta, \tau | \mathbf{X}, \mathbf{Y}) \propto L(\beta, \tau | \mathbf{X}, \mathbf{Y}) \pi(\beta) \pi(\tau).$$

La expresión anterior tiene forma cerrada únicamente bajo algunas condiciones en las distribuciones *a priori*.

Una posibilidad para las distribuciones a priori es asumir que

$$\beta \sim N(c_0, \mathbf{V}_0), \quad \tau \sim \Gamma(a_0, b_0),$$

donde c_0 y $\mathbf{V}_0$ son conocidas. Si $\mathbf{V}_0$ es diagonal entonces se asumen distribuciones previas independientes para β . Los parámetros de la gamma son ambos conocidos (esta parametrización asume que $E(\tau) = a_0/b_0$ y $Var(\tau) = a_0/b_0^2$).

Estimacion Bayesiana en el modelo lineal

Específicamente:

Para hacer inferencias acerca de los coeficientes, requerimos saber distribución a priori de β , y τ .

PRIORIS (No-informativas)

- $\beta_i \sim Normal(0, 10^{-6})$
- $\sigma^2 \sim GamaInversa(.0001, .0001)$
- $\tau = 1/\sigma^2 \sim Gamma(.0001, .0001)$

Esta priori es buena cuando tenemos muchos datos y pocos parámetros.

Regresión clasica en R

Un ejemplo biologico (Olivier Gimenez) Cigüeña blanca(Ciconia ciconia) en Alsace 1948-1970
componentes Demograficos(fecundidad, reproduccion, supervivencia, etc...) explicados por factores ambientales. **Relación de interereres numero de polluelos promedio por nido explicado por temperatura y precipitación el mes de mayo**

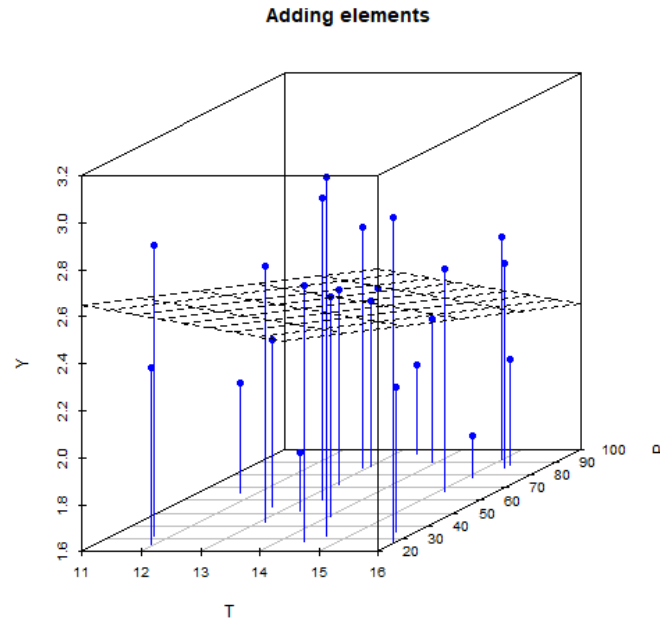


T	P	Y
15.1	67	2.55
13.3	52	1.85
15.3	88	2.05
13.3	61	2.88
14.6	32	3.13
15.6	36	2.21
13.1	72	2.43
13.1	43	2.69
15.0	92	2.55
11.7	32	2.84
15.3	86	2.47
14.4	28	2.69
14.4	57	2.52
12.7	55	2.31
11.7	66	2.07
11.9	26	2.35
15.9	28	2.98
13.4	96	1.98
14.0	48	2.53
13.9	90	2.21
12.9	86	2.62
15.1	78	1.78
13.3	57	2.33

Regresión clásica en R

$$Y = \alpha + \beta_1(T) + \beta_2(P) + \epsilon$$

term	estimate	std.error	statistic	p.value
(Intercept)	2.4510457	0.7650790	3.2036505	0.0044589
T	0.0310692	0.0546897	0.5680999	0.5762904
P	-0.0073157	0.0028967	-2.5254812	0.0201073



Regresion Bayesina en *rstanarm* de R

Usa **Stan** (a través del paquete `rstan`) para la estimación bayesiana en personas iniciando en la estadística Bayesiana.

Ajustar modelos con `rstanarm` también es útil para usuarios experimentados de software bayesiano.

Stan es un lenguaje para el análisis de datos bayesiano. El trabajo real se realiza en C++, pero el lenguaje Stan especifica los aspectos necesarios del modelo. Utiliza una variedad de procedimientos de inferencia, incluidas las técnicas de optimización estándar, pero el enfoque principal específico del bayesiano se refiere a Hamiltonian Monte Carlo⁵⁵ Originalmente llamado Hybrid Monte Carlo. de formas en que puede llevarse a cabo la estimación/muestreo bayesiano y este es uno que tiene, como los otros, ventajas en algunas áreas y desventajas en muchos tipos de problemas, en relación con Gibbs y algunos otros muestreadores, HMC será más eficiente en el sentido de que se necesitarán menos iteraciones para describir la distribución posterior. Sin embargo, es aplicable en una amplia variedad de problemas y, a menudo, funcionará mejor que otros enfoques.

Usando Stan

```
library(mlbench)
library(rstanarm)
library(bayestestR)
library(bayesplot)
library(insight)
library(broom)
polluelos <- read.csv("polluelos.csv")
model_bayes<- stan_glm(Y~T+P, data=polluelos, seed=111)
```

```
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 7.2e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.72 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 2000 [  0%] (Warmup)
## Chain 1: Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 1: Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 1: Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 1: Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 1: Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 1: Iteration: 1001 / 2000 [ 50%] (Sampling)
```

```
print(model_bayes, digits = 3)
```

```
## stan_glm
## family:      gaussian [identity]
## formula:     Y ~ T + P
## observations: 23
## predictors:  3
## -----
##              Median MAD_SD
## (Intercept)  2.448  0.733
## T            0.031  0.053
## P           -0.007  0.003
##
## Auxiliary parameter(s):
##           Median MAD_SD
## sigma 0.327  0.052
##
## -----
## * For help interpreting the printed output see ?print.stanreg
## * For info on the priors used see ?prior_summary.stanreg
```

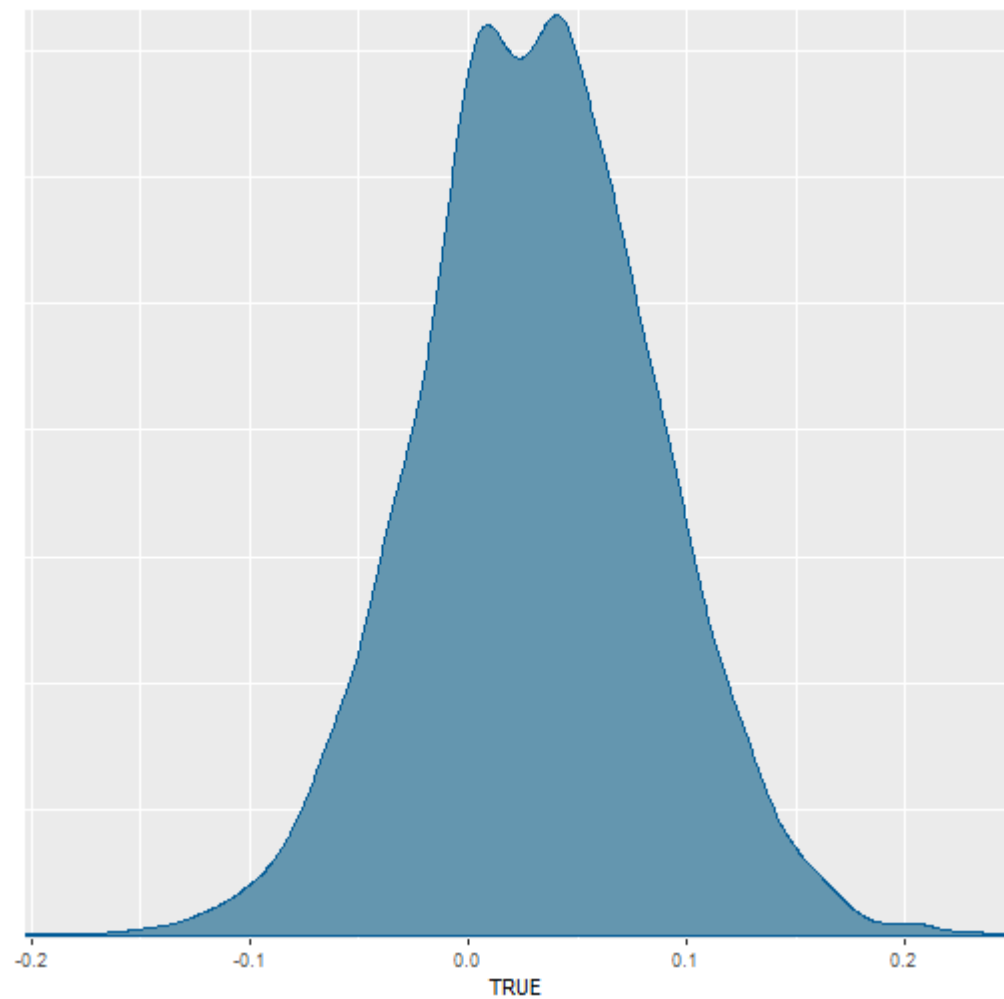
```
describe_posterior(model_bayes)
```

```
## Summary of Posterior Distribution
```

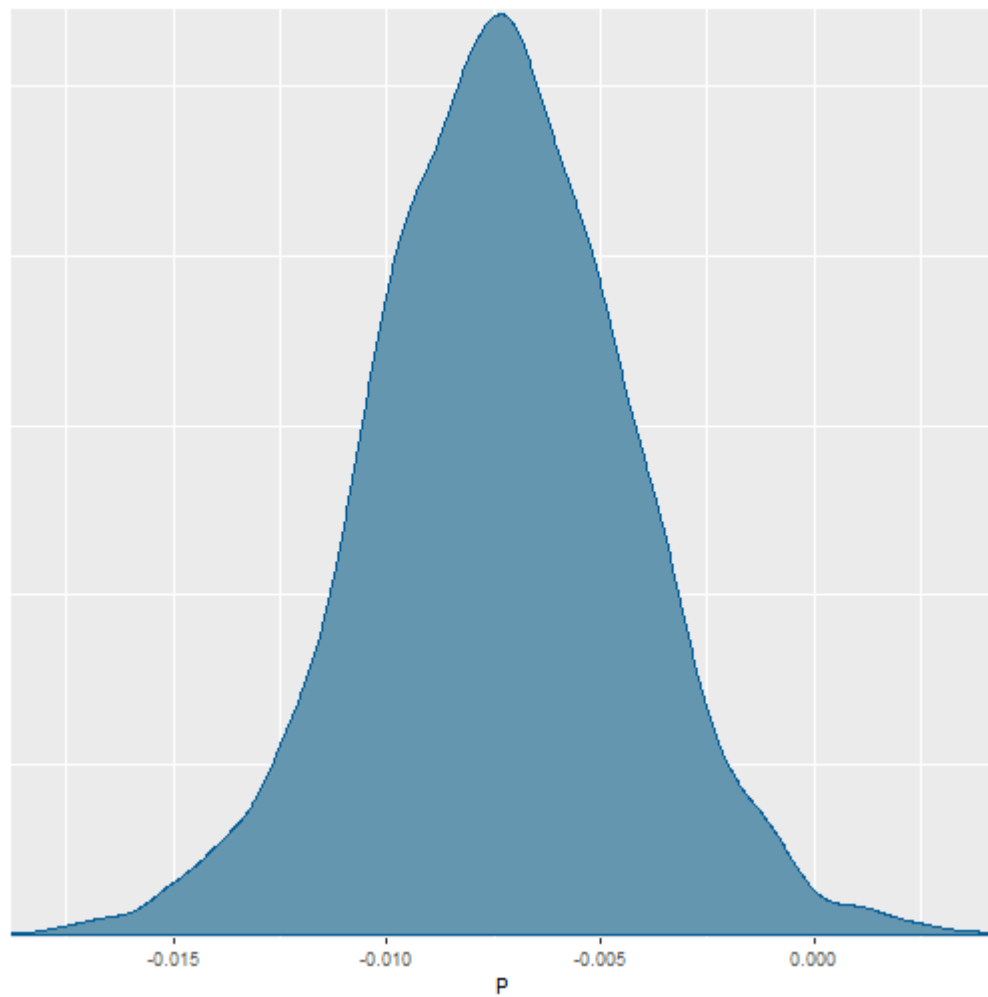
```
##
```

## Parameter	Median	95% CI	pd	ROPE	% in ROPE	Rhat	ESS
##	-----	-----	-----	-----	-----	-----	-----
## (Intercept)	2.45	[0.95, 3.94]	99.80%	[-0.04, 0.04]	0%	0.999	3277.00
## T	0.03	[-0.08, 0.14]	72.42%	[-0.04, 0.04]	44.32%	0.999	3289.00
## P	-7.37e-03	[-0.01, 0.00]	98.98%	[-0.04, 0.04]	100%	1.002	3570.00

```
mcmc_dens(model_bayes, pars = c("T"))
```



```
mcmc_dens(model_bayes, pars=c("P"))
```



```
hdi(model_bayes)
```

```
## Highest Density Interval
```

```
##
```

```
## Parameter      |          95% HDI
```

```
## -----
```

```
## (Intercept) | [ 1.06,  4.02]
```

```
## T           | [-0.08,  0.13]
```

```
## P           | [-0.01,  0.00]
```

```
eti(model_bayes)
```

```
## Equal-Tailed Interval
```

```
##
```

```
## Parameter      |          95% ETI | Effects | Component
```

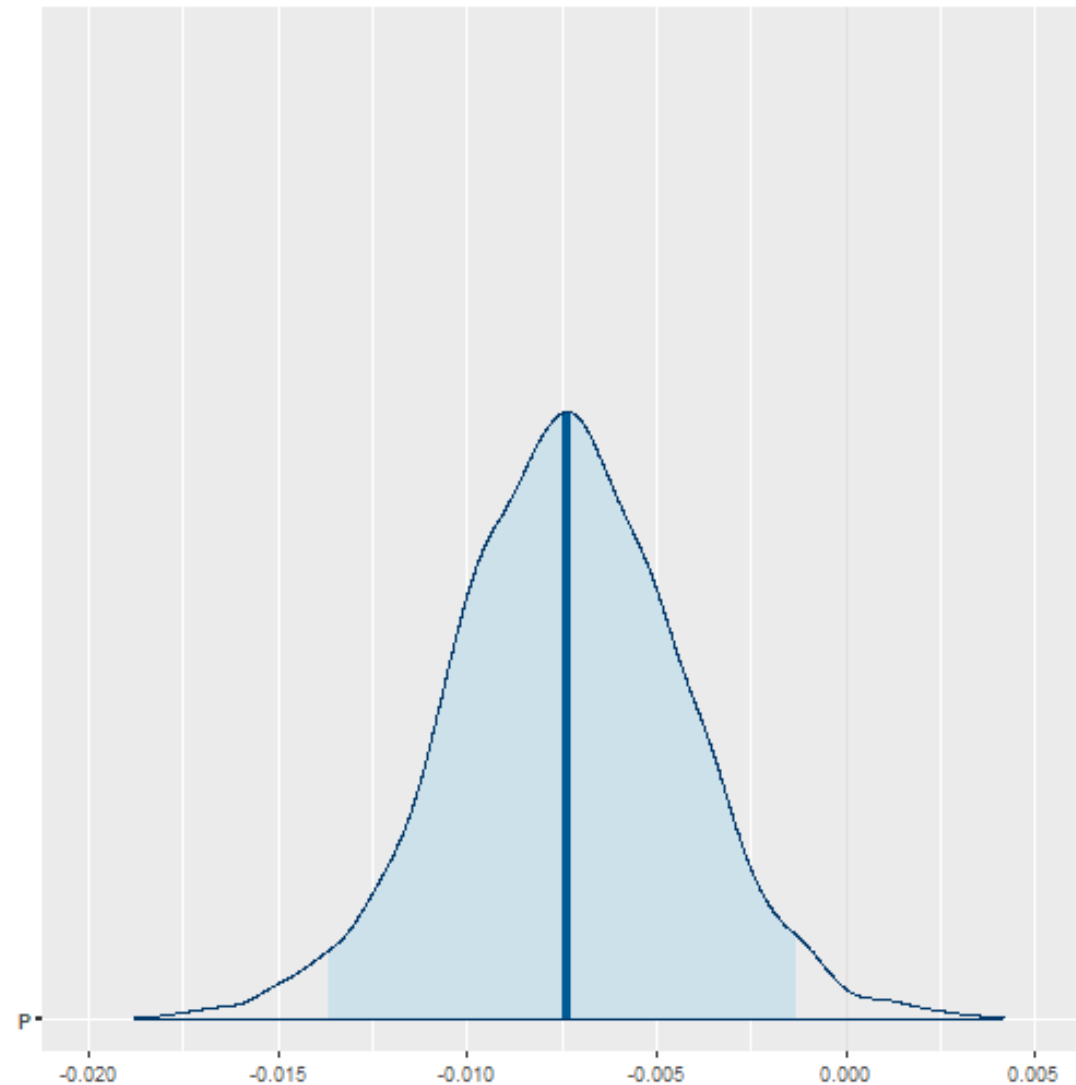
```
## -----
```

```
## (Intercept) | [ 0.95,  3.94] | fixed  | conditional
```

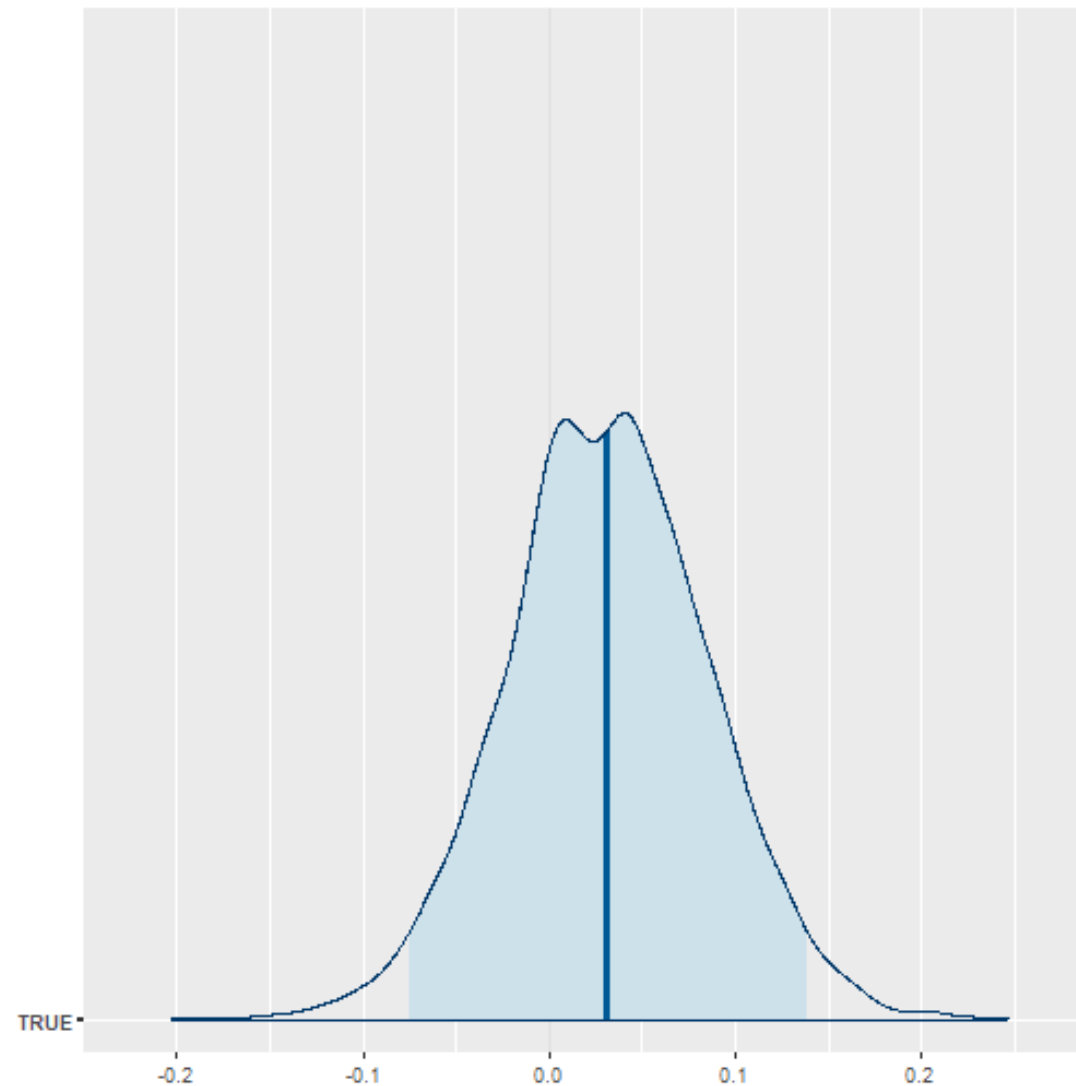
```
## T           | [-0.08,  0.14] | fixed  | conditional
```

```
## P           | [-0.01,  0.00] | fixed  | conditional
```

Distribucion Posterior de Precipitacion



Distribución Posterior de Temperatura



Integrated Nested Laplace Approximation (INLA)

Otro enfoque para lograr la inferencia bayesiana ha sido propuesto por Havard Rue, Martino y Chopin (2009), quienes a través de métodos computacionales estiman las distribuciones a posteriori marginales para cada uno de los parámetros en el modelo. Es decir, en lugar de estimar la distribución a posteriori conjunta $\pi(\theta|\mathbf{y})$ (distribución multivariada de alta multidimensionalidad), INLA estima las distribuciones marginales a posteriori de cada parámetro del modelo $\pi(\theta_i|\mathbf{y})$.

A diferencia de MCMC que usa métodos de simulación, INLA usa métodos de aproximación para el ajuste del modelo bayesiano. Dentro de la clase de LGM, INLA puede adaptarse a modelos mucho más rápido que los métodos basados en MCMC.

Rue, H., Martino, S. and Chopin, N. (2009), Approximate Bayesian inference for latent Gaussian models by using integrated nested Laplace approximations. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 71: 319-392. <https://doi.org/10.1111/j.1467-9868.2008.00700.x>

Usando la libreria *brms* de R

Como alternativa al uso directo de *stan* se tiene el paquete *brms* (Bürkner, 2017b), que implementa BMLM en R. El paquete *brms* tiene un objetivo similar al paquete *rstanarm*: proporcionar una interfaz de usuario simple para STAN. Sin embargo, a diferencia de la implementación de *rstanarm*, *brms* simplemente convierte la fórmula, los datos, las prioris y la familia en código y datos del modelo STAN antes de ejecutar *stan* con esos elementos.

Si bien no es necesario especificar los valores previos al usar las funciones de *brms* (ya que se generarán valores predeterminados), no hay garantía de que las rutinas para determinar estos valores predeterminados persistan en el tiempo. Además, siempre es mejor definir tus propias prioris si no es por otra razón que te obliga a pensar en lo que estás haciendo.

Bürkner PC (2017). "brms: An R Package for Bayesian Multilevel Models using Stan", *Journal of Statistical Software*.

Usando la libreria *brms* de R

```
library(brms)
library(broom)
library(broom.mixed)
library(coda)
previas <- c(
  prior(normal(0, 10), class = Intercept),
  prior(normal(0, 100), class = b, coef = T),
  prior(normal(0, 100), class = b, coef = P),
  prior(cauchy(0, 10), class = sigma)
)
bmod1 <- brm( Y~T+P, data=polluelos, family = gaussian(),
  prior = previas, warmup = 2000, iter = 5000
)
```

```
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 3.6e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.36 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 5000 [  0%]   (Warmup)
## Chain 1: Iteration:   500 / 5000 [ 10%]   (Warmup)
```

Usando la libreria *brms* de R

```
knitr::kable(tidy(bmod1, conf.int = TRUE, conf.method = "HPDinterval"))
```

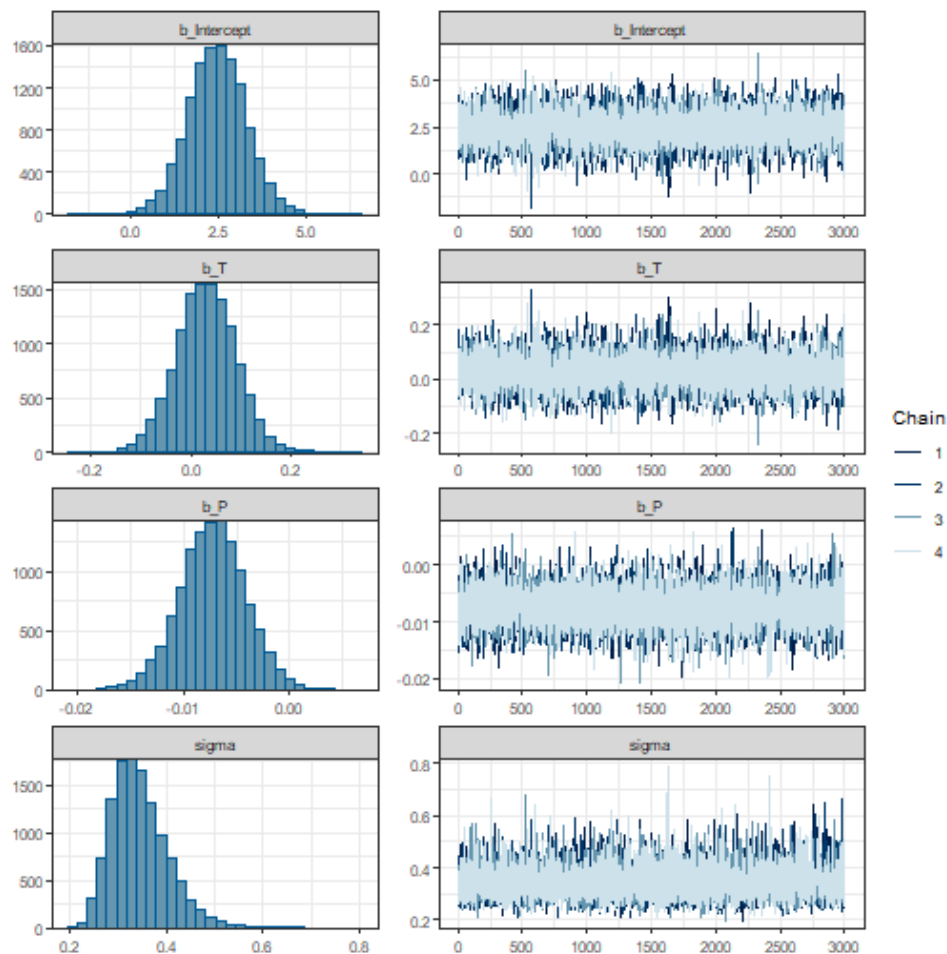
effect	component	group	term	estimate	std.error	conf.low	conf.high
fixed	cond	NA	(Intercept)	2.4478009	0.8251938	0.8152148	4.0718428
fixed	cond	NA	T	0.0313468	0.0592792	-0.0817153	0.1516707
fixed	cond	NA	P	-0.0073238	0.0031670	-0.0136144	-0.0011332
ran_pars	cond	Residual	sd__Observation	0.3449035	0.0582782	0.2437984	0.4631211

```
knitr::kable(bayes_R2(bmod1))
```

	Estimate	Est.Error	Q2.5	Q97.5
R2	0.2602657	0.1213029	0.0317522	0.4754665

Usando la libreria *brms* de R

```
library(tidyverse)
bmod1 %>% plot(combo = c("hist", "trace"), widths = c(1, 1.5),
  theme=theme_bw(base_size=10))
```

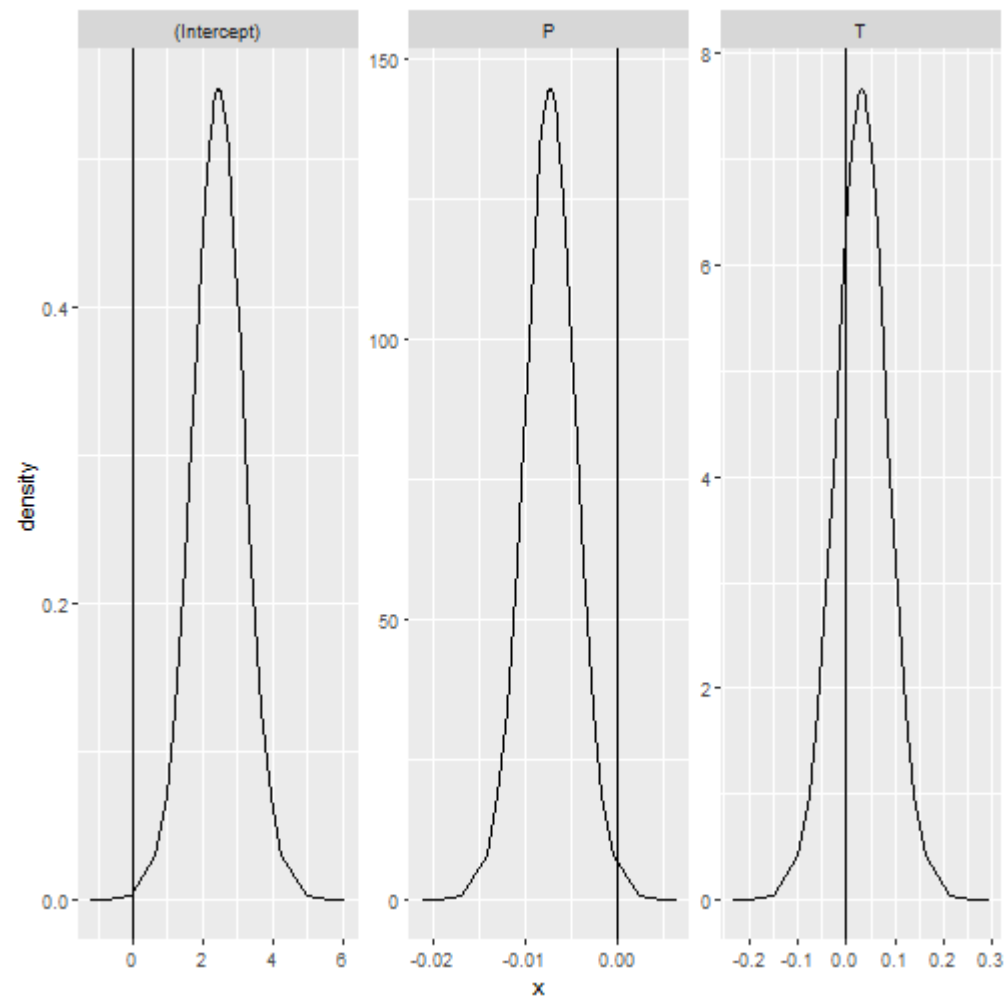


en INLA

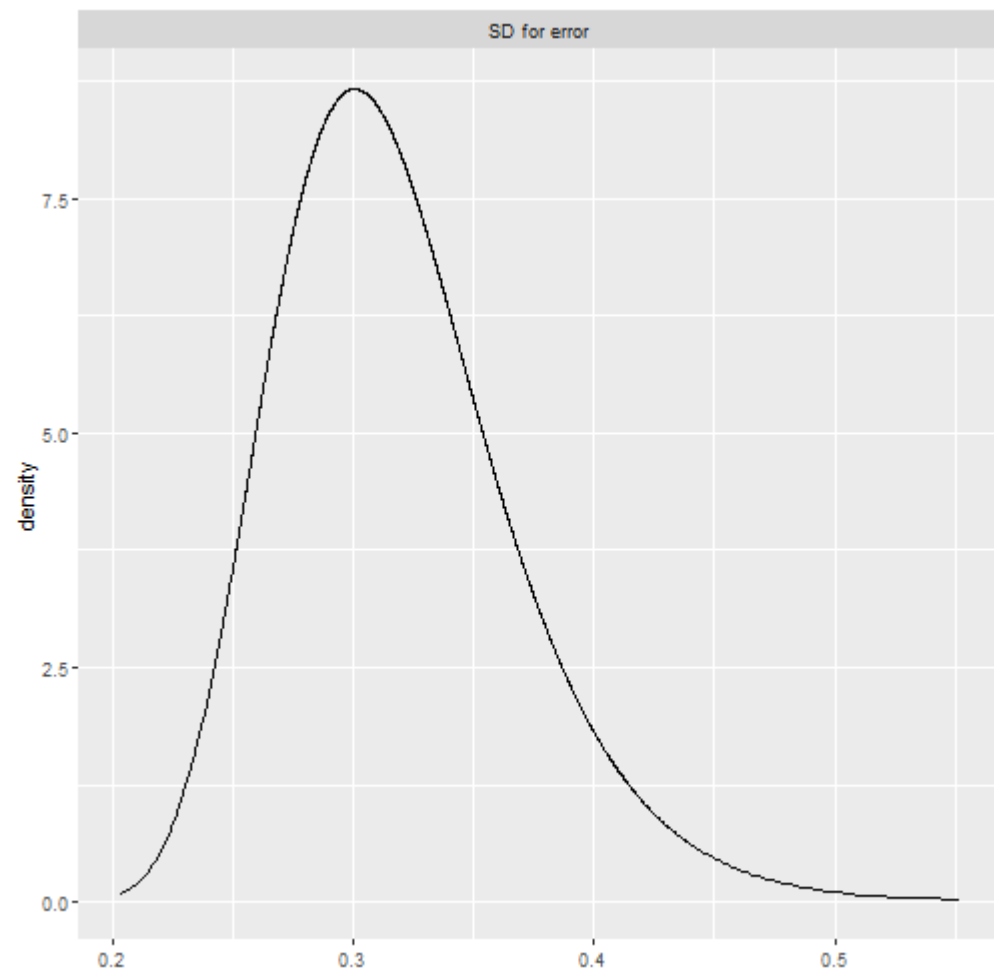
```
library(INLA)
library(gridExtra)
library(ggplot2)
library(brinla)
inlamod <- inla(Y~T+P, data=polluelos, family="gaussian")
inlamod$summary.fixed[,1:2]
```

##	mean	sd
## (Intercept)	2.451046967	0.758492138
## T	0.031069112	0.054218837
## P	-0.007315652	0.002871801

```
bri.fixed.plot(inlamod, together = FALSE)
```



```
bri.hyperpar.plot(inlamod, together = FALSE)
```



```
summary(inlamod)
```

```
##
## Call:
##      c("inla.core(formula = formula, family = family, contrasts = contrasts,
##      ", " data = data, quantiles = quantiles, E = E, offset = offset, ", "
##      scale = scale, weights = weights, Ntrials = Ntrials, strata = strata,
##      ", " lp.scale = lp.scale, link.covariates = link.covariates, verbose =
##      verbose, ", " lincomb = lincomb, selection = selection, control.compute
##      = control.compute, ", " control.predictor = control.predictor,
##      control.family = control.family, ", " control.inla = control.inla,
##      control.fixed = control.fixed, ", " control.mode = control.mode,
##      control.expert = control.expert, ", " control.hazard = control.hazard,
##      control.lincomb = control.lincomb, ", " control.update =
##      control.update, control.lp.scale = control.lp.scale, ", "
##      control.pardiso = control.pardiso, only.hyperparam = only.hyperparam,
##      ", " inla.call = inla.call, inla.arg = inla.arg, num.threads =
##      num.threads, ", " keep = keep, working.directory = working.directory,
##      silent = silent, ", " inla.mode = inla.mode, safe = FALSE, debug =
##      debug, .parent.frame = .parent.frame)" )
## Time used:
##      Pre = 1.85, Running = 0.303, Post = 0.0716, Total = 2.22
## Fixed effects:
##              mean      sd 0.025quant 0.5quant 0.975quant   mode kld
## (Intercept)  2.451 0.758      0.950    2.451      3.952  2.451  0
## T            0.031 0.054     -0.076    0.031      0.138  0.031  0
## D            0.007 0.003      0.012    0.007      0.002  0.007  0
```

sas proc MCMC

```
data uno;
input num_poll temp prec;
cards;
2.55      15.1      67
1.85      13.3      52
2.05      15.3      88
2.88      13.3      61
3.13      14.6      32
2.21      15.6      36
2.43      13.1      72
2.69      13.1      43
2.55      15        92
2.84      11.7      32
2.47      15.3      86
2.69      14.4      28
2.52      14.4      57
2.31      12.7      55
2.07      11.7      66
2.35      11.9      26
2.98      15.9      28
1.98      13.4      96
2.53      14        48
2.21      13.9      90
2.62      12.9      86
1.78      15.1      78
2.3       13        87
;
proc mcmc data=uno outpost=salida nmc=50000 thin=5 seed=246810;
  parms beta0 0 beta1 0 beta2 0;
  parms sigma2 1;
  prior beta0 beta1 beta2 ~ normal(mean = 0, var = 1e6);
  prior sigma2 ~ igamma(shape = 3/10, scale = 10/3);
  mu = beta0 + beta1*temp +beta2*prec;
  model num_poll ~ n(mu, var = sigma2);
run;
```

ANOVA Bayesiano

Ejemplo: Experimento agrícola DCA

Daño de insectos en Trigo

Para evaluar el variación del *grado de daño* causado por insectos sobre diferentes *variedades* de trigo (Milliken y Johnson 2009). El conjunto de datos se da en la Tabla 1. El experimentador seleccionado al azar cuatro variedades de trigo de una gran cantidad de variedades y realizó un experimento para evaluar el daño causado por insectos en las plantas de trigo justo antes de del espigamiento. La estructura del diseño fue un *diseño completamente al azar* con cuatro repeticiones o parcelas por variedad. (DCA). Debido a las condiciones ambientales, algunas de las parcelas fueron destruidas (exceso de lluvia). Por lo tanto, el diseño también fue un diseño desbalanceado. Se seleccionaron al azar 20 plantas de cada parcela disponible y calificó la cantidad de daño causado por insectos a cada planta usando un escala de 0 a 10 donde 0 indica que no hay daño y 10 indica daño severo. Por tanto, la respuesta medida en cada parcela es la media de las calificaciones de los 20 plantas dentro de cada parcela.

Plot	Wheat	Damage
1	A	3.9
2	A	4.05
3	A	4.25
4	B	3.6
5	B	4.2
6	B	4.05
7	B	3.85
8	C	4.15
9	C	4.6
10	C	4.15
11	C	4.4
12	D	3.35
13	D	3.8

X. Che and S. Xu.(2011). Bayesian data analysis for agricultural experiments. Canadian Journal of Plant Science. 90(5): 575-603. <https://doi.org/10.4141/CJPS10004>

Enfoque clasico

Modelo aditivo de una vía de clasificación:

$$y_{ij} = \mu + \tau_i + \epsilon_{ij}.$$

Los parámetros τ_i se interpretan como el efecto del tratamiento i-ésimo. μ es la media general.

Se supone de que los errores ϵ_{ij} son independientes e idénticamente distribuidos (iid) con varianza común $Var(\epsilon_{ij}) = \sigma^2$, para todo i, j entonces $E(CM_{Trat}) = \sum_{i=1}^a \tau_i^2 + \sigma^2$, $E(CME) = \sigma^2$. En forma matricial:

$$\begin{pmatrix} 3.9 \\ 4.05 \\ 4.25 \\ 3.6 \\ 4.2 \\ 4.05 \\ 3.85 \\ 4.15 \\ 4.6 \\ 4.15 \\ 4.4 \\ 3.35 \\ 3.8 \end{pmatrix} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} \mu \\ \tau_1 \\ \tau_2 \\ \tau_3 \\ \tau_4 \end{pmatrix} + \begin{pmatrix} \epsilon_{1,1} \\ \epsilon_{1,2} \\ \epsilon_{1,3} \\ \epsilon_{2,1} \\ \epsilon_{2,2} \\ \epsilon_{2,3} \\ \epsilon_{2,4} \\ \epsilon_{3,1} \\ \epsilon_{3,2} \\ \epsilon_{3,3} \\ \epsilon_{3,4} \\ \epsilon_{4,1} \\ \epsilon_{4,2} \end{pmatrix},$$

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}.$$

Librerías y datos

```
library(conflicted)
library(multcomp)
library(multcompView)
library(emmeans)
dca=data.frame(Damage=as.matrix(c(3.9,4.05,4.25,3.6,4.2,4.05,3.85,4.15,4.6,4.15,4.4,3.35,3.8)),
Wheat = factor(c(rep("A",3),rep("B",4),rep("C",4),rep("D",2))))
dca <- dca %>%
mutate_at(vars( Wheat), as.factor)
```

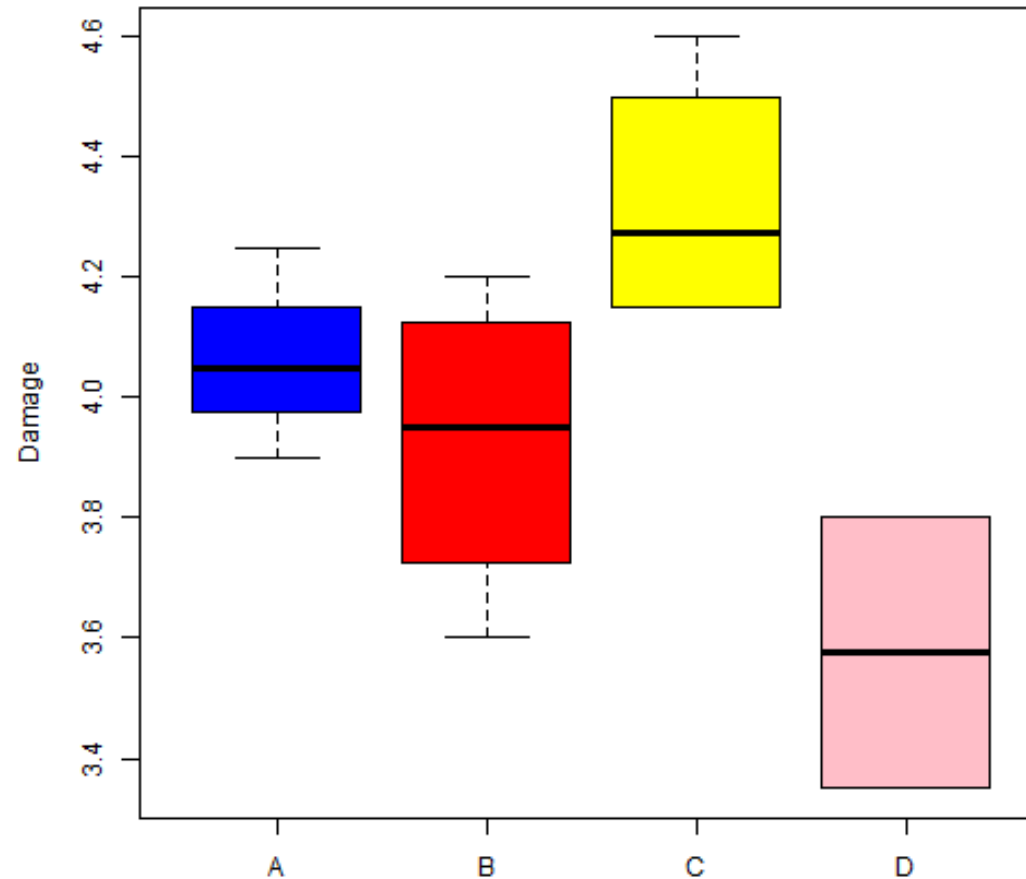
Calculo de medias y desviacion estandar por tratamiento

```
library(tidyverse)
medias=dca %>%
  group_by(Wheat) %>%
  summarize(mean = mean(Damage),
            std.dev = sd(Damage))
knitr::kable(medias)
```

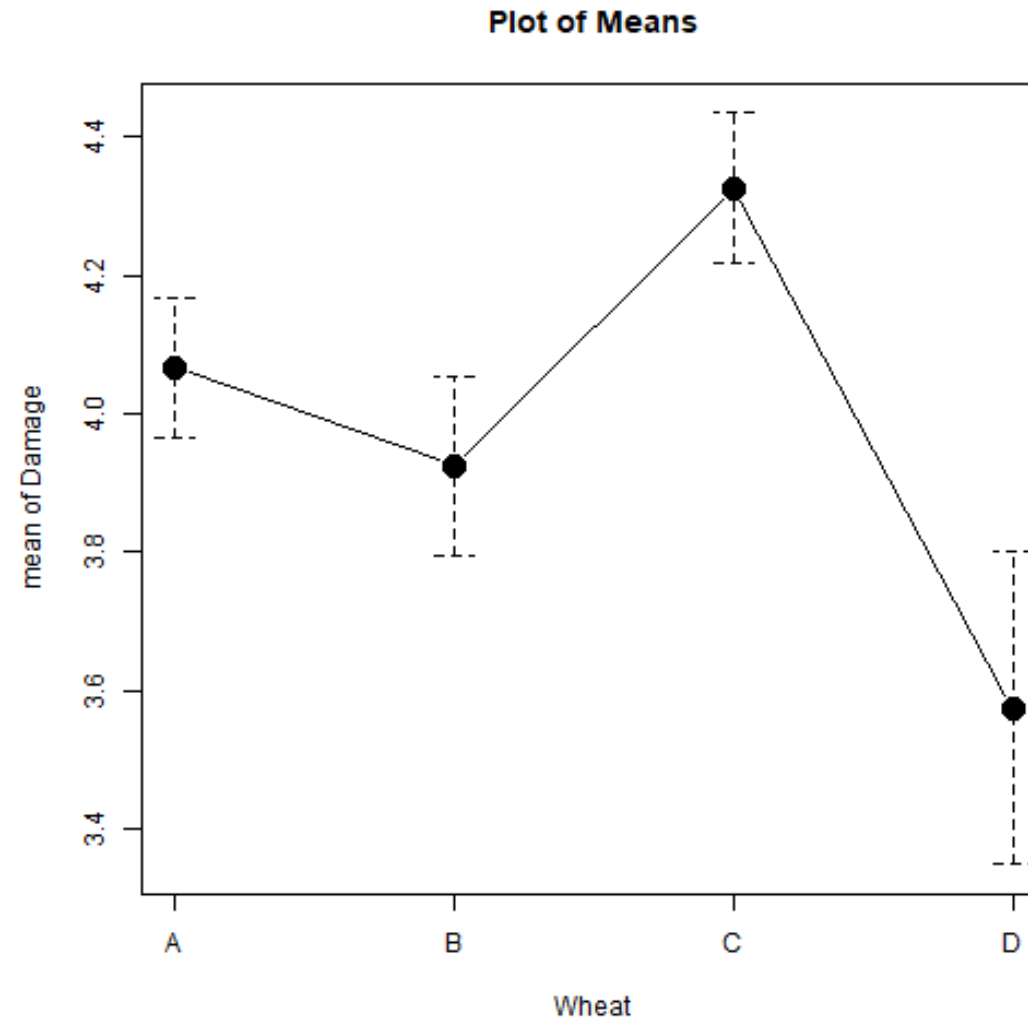
Wheat	mean	std.dev
A	4.066667	0.1755942
B	3.925000	0.2598076
C	4.325000	0.2179449
D	3.575000	0.3181981

Grafica de Cajas

```
boxplot(Damage~Wheat, data =dca,col=c("blue", "red", "yellow", "pink"))
```



Grafica de medias con error estandar



```
mod <- lm(Damage ~ Wheat, data = dca)
library(equatiomatic)
# Modelo ajustado
extract_eq(mod, wrap = TRUE, ital_vars = TRUE)
```

$$Damage = \alpha + \beta_1(Wheat_B) + \beta_2(Wheat_C) + \beta_3(Wheat_D) + \epsilon$$

```
aov(mod)
```

```
## Call:
##      aov(formula = mod)
##
## Terms:
##              Wheat Residuals
## Sum of Squares  0.8101603 0.5079167
## Deg. of Freedom      3      9
##
## Residual standard error: 0.2375609
## Estimated effects may be unbalanced
```

Comparacion de Medias Tukey

```
mean_comparisons <- mod %>%
  emmeans(pairwise ~ "Wheat", adjust="tukey") %>%
  pluck("emmeans") %>%
  cld(details=TRUE, Letters=letters) # add letter display
mean_comparisons$emmeans # adjusted variety means
```

```
## Wheat emmean SE df lower.CL upper.CL .group
## D 3.58 0.168 9 3.20 3.95 a
## B 3.92 0.119 9 3.66 4.19 ab
## A 4.07 0.137 9 3.76 4.38 ab
## C 4.33 0.119 9 4.06 4.59 b
##
## Confidence level used: 0.95
## P value adjustment: tukey method for comparing a family of 4 estimates
## significance level used: alpha = 0.05
## NOTE: If two or more means share the same grouping symbol,
## then we cannot show them to be different.
## But we also did not show them to be the same.
```

```
mean_comparisons$comparisons
```

```
## contrast estimate SE df t.ratio p.value
## B - D 0.350 0.206 9 1.701 0.3767
## A - D 0.492 0.217 9 2.267 0.1772
## A - B 0.142 0.181 9 0.781 0.8613
## C - D 0.750 0.206 9 3.645 0.0228
## C - B 0.400 0.168 9 2.381 0.1505
## C - A 0.258 0.181 9 1.424 0.5166
```

Enfoque Bayesiano

El modelo ANOVA es un caso particular del modelo lineal: Modelo de *_celdas medias* (reparametrizado)

$$y_{ij} = \theta_i + \epsilon_{ij}.$$

Suponiendo que $\epsilon_{ij} \sim n(0, 1/\phi)$, *iid* para $i = 1, \dots, k$, y $j = 1, \dots, n_i$

Los parámetros θ_i the parameters are $\theta = (\theta_1, \theta_2, \dots, \theta_k)$. con

$$y = (y_{11}, \dots, y_{1n_1}, y_{21}, \dots, y_{2n_2}, \dots, y_{k1}, \dots, y_{kn_k})^t$$

$$X = \begin{bmatrix} 1 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 1_{n_1} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 1_{n_2} & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

usando *prioris* normales condicionalmente independientes, $\theta_i \sim N(m_i, 1/\alpha_i\phi)$, para $i = 1 \dots k$, y un una *priori* gamma $\phi \sim G(a/2, b/2)$ cono $\sigma^2 = 1/\phi$.

Usando MCMCpack

MCMCpack (Martin, Quinn y Park 2011) es un paquete de R que contiene funciones para realizar inferencia bayesiana. Proporciona un entorno informático con las herramientas bayesianas (en particular los métodos MCMC) fácil de usar. MCMCpack contiene actualmente los dieciocho modelos estadísticos: modelos de regresión lineal (regresión lineal con errores gaussianos, regresión de descomposición de valor singular y una regresión para una variable dependiente censurada), modelos de elección discreta (regresión logística, regresión multinomial logística, regresión probit ordinal y regresión probit), etc.

Martin, A., Quinn, K., & Park, J. (2011). MCMCpack: Markov Chain Monte Carlo in R. *Journal of Statistical Software*, 42(9), 1 - 21. doi:<http://dx.doi.org/10.18637/jss.v042.i09>

Ajuste del modelo en MCMCpack

```
library(MCMCpack)
library(tidyverse)
library(brms)
library(MCMCpack)
library("tidymodels")
library(broom.mixed)
library(broom)
```

```
trigo.mcmcpack = MCMCregress(Damage ~-1+ Wheat, data = dca)
summary(trigo.mcmcpack)
```

```
##
## Iterations = 1001:11000
## Thinning interval = 1
## Number of chains = 1
## Sample size per chain = 10000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## WheatA 4.06799 0.15625 0.0015625      0.0015287
## WheatB 3.92627 0.13618 0.0013618      0.0013808
## WheatC 4.32526 0.13200 0.0013200      0.0012886
## WheatD 3.57591 0.18541 0.0018541      0.0018541
## sigma2 0.07217 0.04454 0.0004454      0.0006592
##
## 2. Quantiles for each variable:
```

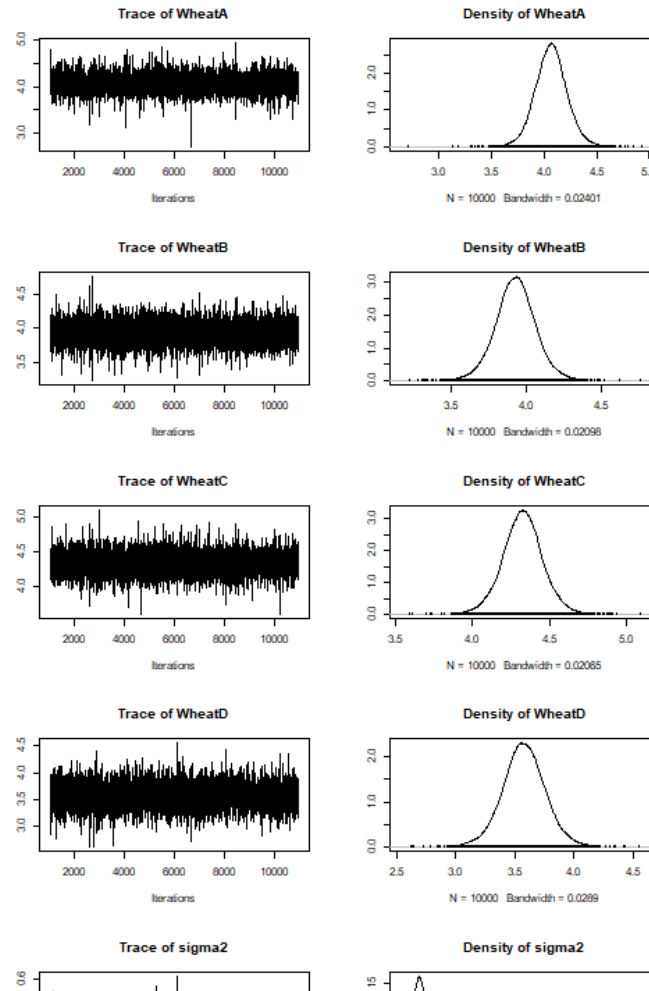
Intervalos de credibilidad

```
knitr::kable(tidyMCMC(trigo.mcmcpack, conf.int = TRUE, conf.method = "HPDinterval"))
```

term	estimate	std.error	conf.low	conf.high
WheatA	4.0677691	0.1562496	3.7576462	4.372626
WheatB	3.9269716	0.1361843	3.6520988	4.196995
WheatC	4.3255963	0.1320012	4.0513441	4.576478
WheatD	3.5754820	0.1854120	3.2138233	3.954331
sigma2	0.0606153	0.0445411	0.0219142	0.154759

diagnostico de convergencia

```
plot(trigo.mcmcpack)
```



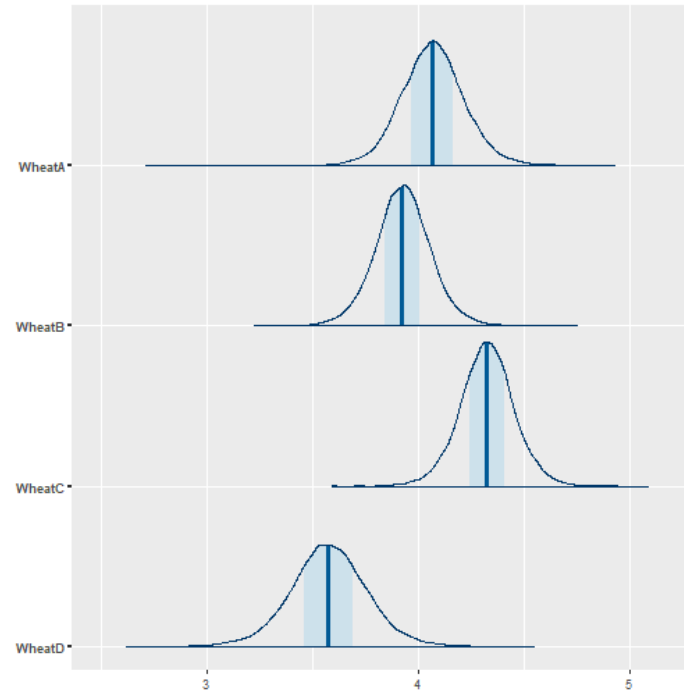
diagnostico de convergencia

```
raftery.diag(trigo.mcmcpack)
```

```
##  
## Quantile (q) = 0.025  
## Accuracy (r) = +/- 0.005  
## Probability (s) = 0.95  
##  
##      Burn-in  Total  Lower bound  Dependence  
##      (M)      (N)    (Nmin)      factor (I)  
##  WheatA 3      4163  3746          1.11  
##  WheatB 2      3994  3746          1.07  
##  WheatC 3      4061  3746          1.08  
##  WheatD 3      4061  3746          1.08  
##  sigma2 2      3897  3746          1.04
```

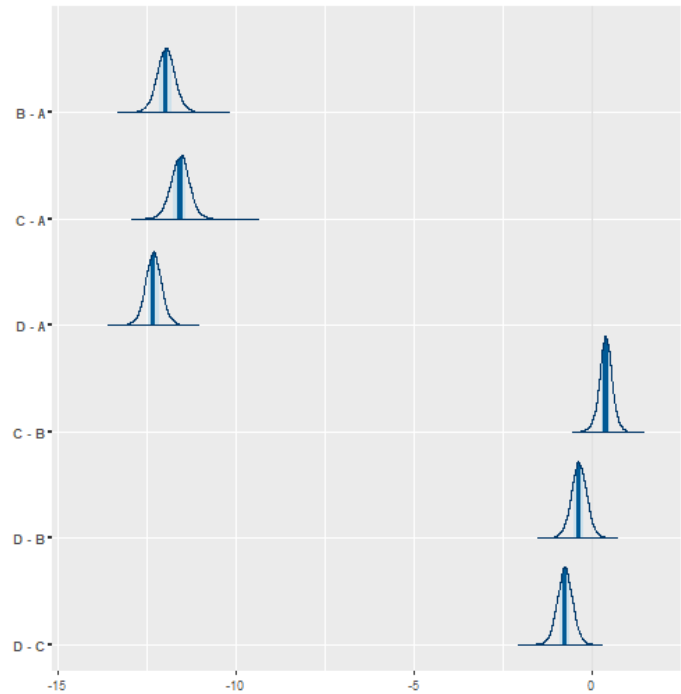
Distribucion posterior efectos Tratamiento

```
library(bayesplot)
#mcmc_intervals(as.matrix(trigo.mcmcpack), regex_pars="Wheat")
#mcmc_areas(as.matrix(trigo.mcmcpack), regex_pars = "Intercept|Wheat|sigma")
mcmc_areas(as.matrix(trigo.mcmcpack), regex_pars = "Wheat")
```



Grafica de intervalos credibilidad

```
day.mcmc = trigo.mcmcpack
wch = grep("Intercept|Wheat", colnames(day.mcmc))
coefs = as.matrix(day.mcmc[, wch])
newdata = data.frame(Wheat = levels(dca$Wheat))
library(multcomp)
tuk.mat <- contrMat(n = table(newdata$Wheat), type = "Tukey")
Xmat2 <- model.matrix(~Wheat, data = newdata)
pairwise.mat <- tuk.mat %*% t(Xmat2)
#pairwise.mat
mcmc_areas(coefs %*% t(pairwise.mat))
```

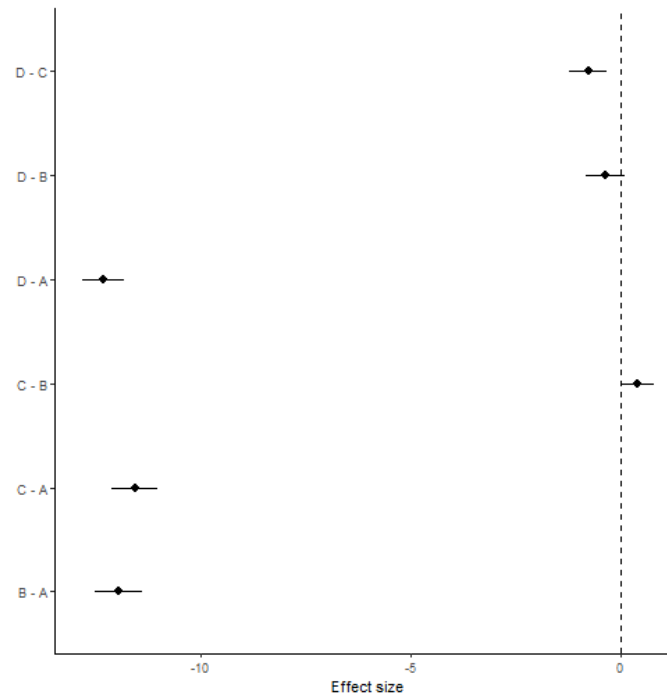


Grafica de intervalos credibilidad

```
(comps = tidyMCMC(coefs %>% t(pairwise.mat), conf.int = TRUE, conf.method = "HPDinterval"))
```

```
## # A tibble: 6 × 5
##   term estimate std.error conf.low conf.high
##   <chr>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 B - A    -12.0         0.275    -12.5      -11.4
## 2 C - A    -11.6         0.280    -12.1      -11.0
## 3 D - A    -12.3         0.245    -12.8      -11.8
## 4 C - B     0.400         0.189     0.0250     0.782
## 5 D - B    -0.353         0.230    -0.811     0.106
## 6 D - C    -0.750         0.228    -1.22     -0.313
```

```
ggplot(comps, aes(y = estimate, x = term)) + geom_pointrange(aes(ymin = conf.low, ymax = conf.high)) +
  geom_hline(yintercept = 0, linetype = "dashed") + scale_y_continuous("Effect size") + scale_x_discrete("") +
  coord_flip() + theme_classic()
```

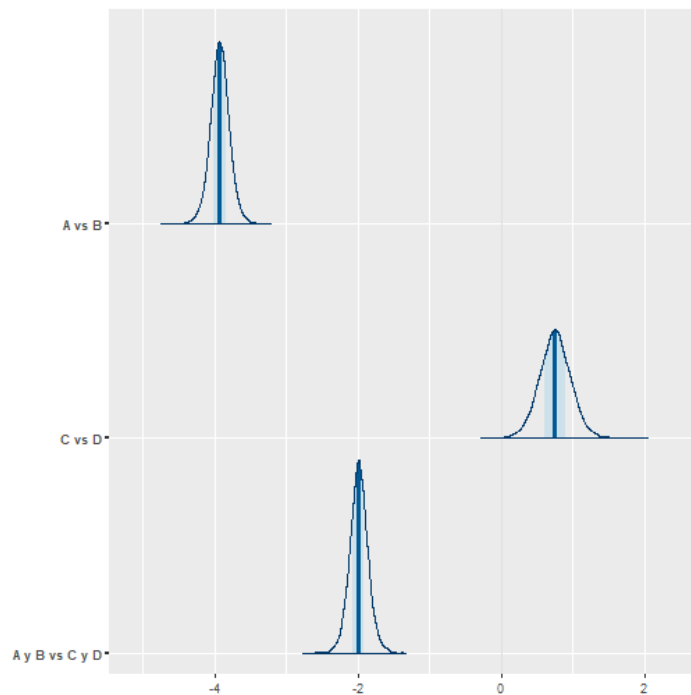


Contrastes Bayesianos

```
# Algunos contrastes
cont.mat = rbind(`A vs B` = c(1, -1, 0, 0), `C vs D` = c(0, 0, 1, -1),
                `A y B vs C y D` = c(1/2, 1/2, -1/2, -1/2))
Xmat2 <- model.matrix(~Wheat, data = newdata)
pairwise.mat1 <- cont.mat %*% Xmat2
pairwise.mat1
```

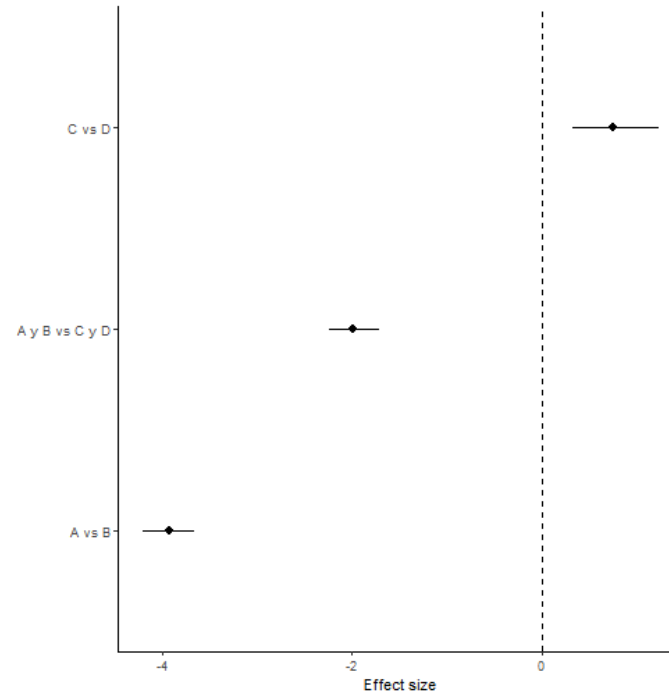
```
##              (Intercept) WheatB WheatC WheatD
## A vs B                0   -1.0    0.0    0.0
## C vs D                0    0.0    1.0   -1.0
## A y B vs C y D        0    0.5   -0.5   -0.5
```

```
mcmc_areas(coefs %*% t(pairwise.mat1))
```



Contrastes Bayesianos

```
ggplot(comps, aes(y = estimate, x = term)) + geom_pointrange(aes(ymin = conf.low, ymax = conf.high)) +  
  geom_hline(yintercept = 0, linetype = "dashed") + scale_y_continuous("Effect size") + scale_x_discrete("") +  
  coord_flip() + theme_classic()
```



INLA ANOVA

Instalación de INLA en R

```
install.packages("INLA",repos=c(getOption("repos"), INLA="https://inla.r-inla-download.org/R/stable"),dep=TRUE)  
devtools::install_github('timcdlucas/INLAutils')
```

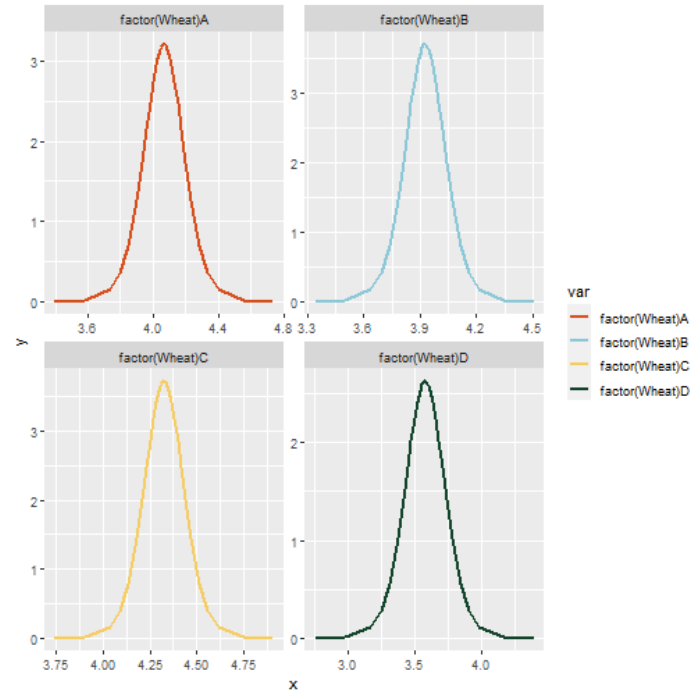
INLA ANOVA

```
require(INLAutils)
require(INLA)
require(sp)
formula <- Damage ~-1+ factor(Wheat)
result <- inla(formula, family="gaussian", data=dca)
result <- inla.hyperpar(result)
summary(result)
```

```
##
## Call:
## c("inla.core(formula = formula, family = family, contrasts = contrasts,
## ", " data = data, quantiles = quantiles, E = E, offset = offset, ", "
## scale = scale, weights = weights, Ntrials = Ntrials, strata = strata,
## ", " lp.scale = lp.scale, link.covariates = link.covariates, verbose =
## verbose, ", " lincomb = lincomb, selection = selection, control.compute
## = control.compute, ", " control.predictor = control.predictor,
## control.family = control.family, ", " control.inla = control.inla,
## control.fixed = control.fixed, ", " control.mode = control.mode,
## control.expert = control.expert, ", " control.hazard = control.hazard,
## control.lincomb = control.lincomb, ", " control.update =
## control.update, control.lp.scale = control.lp.scale, ", "
## control.pardiso = control.pardiso, only.hyperparam = only.hyperparam,
## ", " inla.call = inla.call, inla.arg = inla.arg, num.threads =
## num.threads, ", " keep = keep, working.directory = working.directory,
## silent = silent, ", " inla.mode = inla.mode, safe = FALSE, debug =
## debug, .parent.frame = .parent.frame)" )
## Time used:
## Pre = 1.02, Running = 0.292, Post = 0.0535, Total = 1.36
## Fixed effects:
##          mean      sd 0.025quant 0.5quant 0.975quant  mode kld
## factor(Wheat)A 4.067 0.135      3.798    4.067      4.335 4.067  0
## factor(Wheat)B 3.925 0.117      3.692    3.925      4.157 3.925  0
## factor(Wheat)C 4.325 0.117      4.092    4.325      4.557 4.325  0
## factor(Wheat)D 3.575 0.165      3.246    3.575      3.904 3.575  0
##
## Model hyperparameters:
##          mean      sd 0.025quant 0.5quant
## Precision for the Gaussian observations 21.64 9.22      7.56    20.35
##          0.975quant  mode
## Precision for the Gaussian observations  43.19 17.73
##
## Marginal log-Likelihood: -26.50
## is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

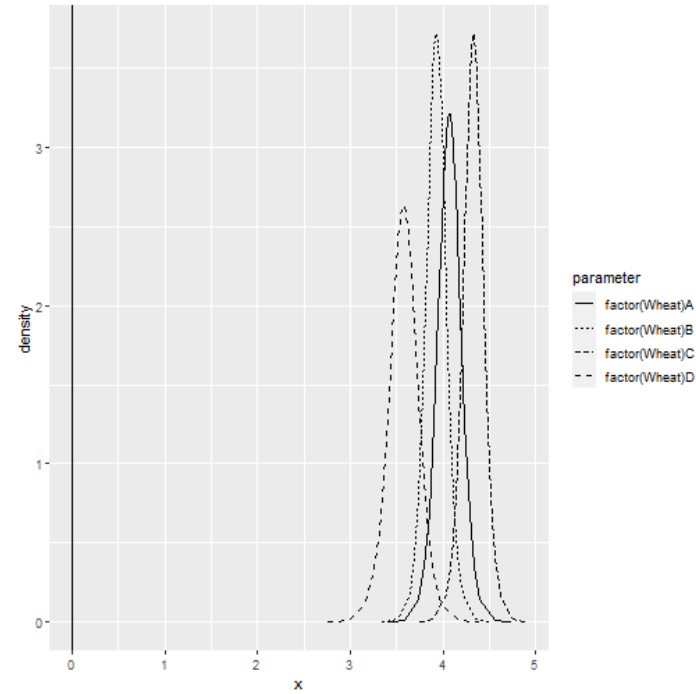
INLA ANOVA

```
require(ggplot2)
require(palettetown) # a convenient palette
p[[1]] + geom_line(aes(colour = var), size = .8) +
  palettetown::scale_colour_poke(pokemon = 'Charmander', spread = 4)
```



INLA ANOVA

```
library(brinla)
bri.fixed.plot(result, together = TRUE)
```



Usando jags

```
modelString = "  
  model {  
    #Likelihood  
    for (i in 1:n) {  
      y[i]~dnorm(mean[i],tau)  
      mean[i] <- inprod(beta[],X[i,])  
    }  
    #Priors  
    for (i in 1:ngroups) {  
      beta[i] ~ dnorm(0, 1.0E-6)  
    }  
    sigma ~ dunif(0, 100)  
    tau <- 1 / (sigma * sigma)  
  }  
"  
  
X <- model.matrix(~ -1+factor(Wheat), data=dca)  
data1.list <- with(dca,list(y=Damage,X=X,n=nrow(dca),ngroups=ncol(X)))  
  
params <- c("beta", "sigma")  
nChains = 3  
burnInSteps = 3000  
thinSteps = 10  
numSavedSteps = 15000 #across all chains  
nIter = ceiling(burnInSteps + (numSavedSteps * thinSteps)/nChains)  
nIter
```

```
## [1] 53000
```

Usando jags

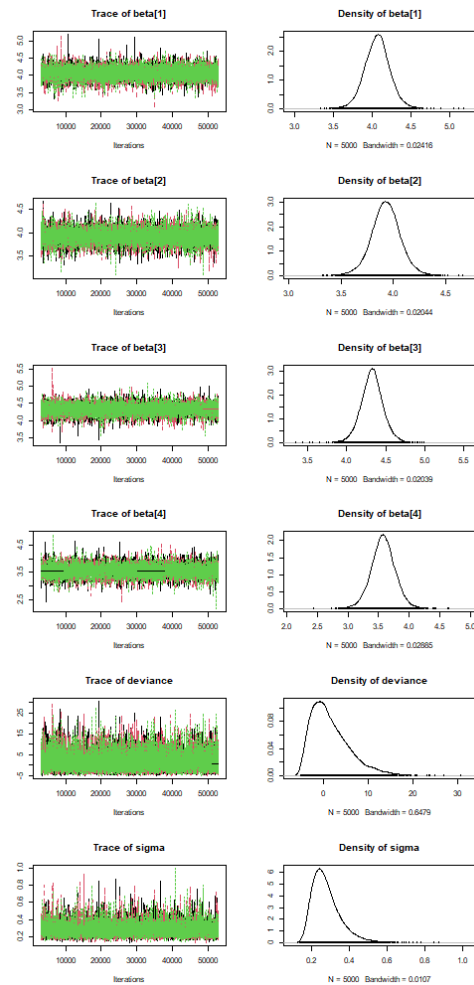
```
library(R2jags)
data1.r2jags <- jags(data = data1.list, inits = NULL, parameters.to.save = params,
model.file = textConnection(modelString),
n.chains = nChains, n.iter = nIter, n.burnin = burnInSteps, n.thin = thinSteps)
```

```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 13
##   Unobserved stochastic nodes: 5
##   Total graph size: 96
##
## Initializing model
```

```
library(coda)
data1.mcmc = as.mcmc(data1.r2jags)
```

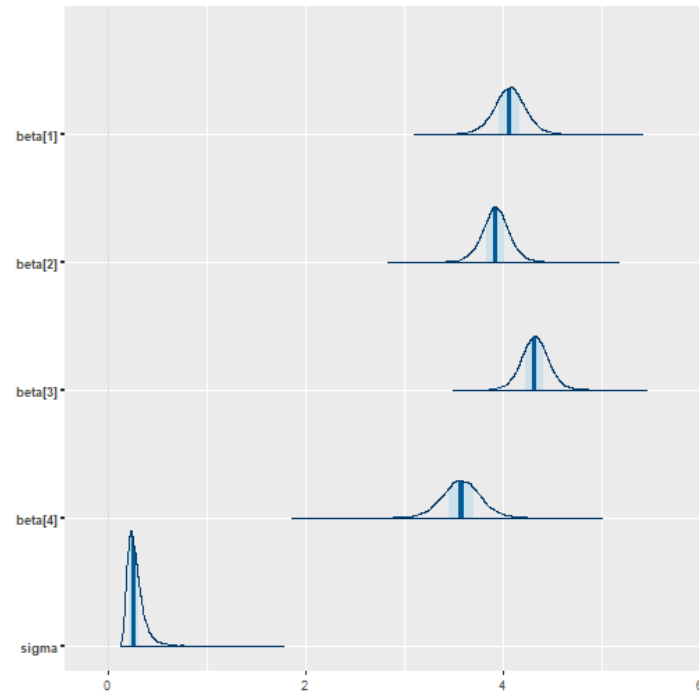
Usando jags

```
plot(data1.mcmc)
```



Usando jags

```
library(bayesplot)
mcmc_areas(data1.r2jags$BUGSoutput$sims.matrix, regex_pars = "beta|sigma")
```



BAYES ANOVA en SAS

```
data trigo;
input plot y x z1-z4;
datalines;
1 3.90 1 1 0 0 0
2 4.05 1 1 0 0 0
3 4.25 1 1 0 0 0
4 3.60 1 0 1 0 0
5 4.20 1 0 1 0 0
6 4.05 1 0 1 0 0
7 3.85 1 0 1 0 0
8 4.15 1 0 0 1 0
9 4.60 1 0 0 1 0
10 4.15 1 0 0 1 0
11 4.40 1 0 0 1 0
12 3.35 1 0 0 0 1
13 3.80 1 0 0 0 1
;
run;
proc mcmc data=trigo outpost=salida2 nmc=50000 thin=5 seed=246810;
  parms beta1 0 beta2 0 beta3 0 beta4 0;
  parms sigma2 1;
  prior beta1 beta2 beta3 beta4~ normal(mean = 0, var = 1e6);
  prior sigma2 ~ igamma(shape = 3/10, scale = 10/3);
  mu = beta1*z1 +beta2*z2+ beta3*z3 +beta4*z4;
  model y ~ n(mu, var = sigma2);
run;
```

Usando brms

```
## library(brms)
## library(tidyverse)
## dca=data.frame(Damage=as.matrix(c(3.9,4.05,4.25,3.6,4.2,4.05,3.85,4.15,4.6,4.15,4.4,3.35,3.8)),
##               Wheat = factor(c(rep("A",3),rep("B",4),rep("C",4),rep("D",2))))
## dca <- dca %>%
##   mutate_at(vars( Wheat), as.factor)
## dca
## trigo_stan1 <- brm(Damage~ -1+Wheat,data = dca,family=gaussian(),seed=1234)
```

Usando brms

```
## trigo_stan1 <- brm(Damage~ -1+Wheat,data = dca,family = gaussian(), seed=1234)  
## trigo_stan1
```

Usando brms

```
## trigo_stan2 <- stan_aov(Damage~Wheat,data = dca,family = gaussian(), seed=1234)  
## trigo_stan2
```